

Double-checked locking for SLURM container tasks

Rohit Goswami · 2026-02-27 · hpc · ci · slurm

When a SLURM job launches N tasks from the same container image, every rank runs the entrypoint in parallel. Commands like `ldconfig` [^fn:1] that write to shared paths (`/etc/ld.so.cache`) collide: multiple tasks try to write the same file simultaneously, the losers get a corrupt or truncated cache, and the job step crashes.

One slightly naive fix involves serialization with `flock`:

```
- flock /tmp/ldconfig.lock -c "ldconfig || true"
```

This stops the crash, but now every rank queues up and runs `ldconfig` in sequence. On a 128-task job, that would add up.

I just needed two things: mutual exclusion (no concurrent writes) and execute-once (only the first rank does the work). `flock` alone gives the first; a marker file gives the second:

```
- flock /tmp/ldconfig.lock -c "test -f /tmp/ld_${CI_JOB_ID} || { ldconfig || true; touch /tmp/ld_${CI_JOB_ID}; }"
```

The first task acquires the lock, runs `ldconfig`, and drops `/tmp/ld_${CI_JOB_ID}`. Every subsequent task acquires the same lock, sees the marker, and exits immediately. Two layers because each solves a different problem: `flock` prevents the race, the marker prevents redundant work.